

DOM XSS in jQuery Selector Sink Using a Hashchange Event

Lab Overview

This lab demonstrates a **DOM-based Cross-Site Scripting (XSS)** vulnerability caused by unsafe handling of data obtained from the URL fragment.

The application uses a jQuery selector as a DOM XSS sink. By manipulating the URL hash and triggering a `hashchange` event, attacker-controlled input can reach the vulnerable selector and execute JavaScript in the victim's browser.

Objective

Exploit the DOM XSS vulnerability and execute JavaScript through the vulnerable jQuery selector.

Vulnerability

Vulnerability Type: DOM-based Cross-Site Scripting (XSS)

The application processes attacker-controlled URL fragment data and passes it into a jQuery selector without appropriate validation or sanitization.

Methodology

1. Open the vulnerable lab application.
2. Inspect the application's behavior and identify functionality that responds to changes in the URL hash.
3. Determine that the hash value is processed by JavaScript after a `hashchange` event.
4. Identify the jQuery selector as the vulnerable DOM sink.
5. Modify the URL fragment with an XSS payload.
6. Trigger the `hashchange` event by changing the hash.
7. Confirm that the injected JavaScript executes in the browser.

Exploitation

The vulnerability can be triggered by placing malicious input in the URL fragment and causing the application to process the modified hash.

Example payload:

```
#<img src=x onerror=alert(document.domain)>
```

The exact payload may require URL encoding depending on how the browser and application process the fragment.

Result

The application processes attacker-controlled input through the vulnerable jQuery selector, resulting in JavaScript execution.

This confirms the presence of a DOM-based XSS vulnerability.

Impact

Successful exploitation could allow an attacker to execute arbitrary JavaScript in the security context of the vulnerable website.

Potential consequences include:

- Theft of sensitive information accessible to JavaScript
- Unauthorized actions performed in the victim's session
- Manipulation of page content
- Phishing attacks
- Session-related attacks where applicable

Remediation

Developers should:

- Avoid passing untrusted input directly into jQuery selectors.
- Validate and constrain URL fragment values before processing them.
- Use safe DOM APIs instead of dynamically constructing selectors from user-controlled data.
- Apply appropriate output encoding where untrusted data is inserted into HTML.
- Consider a strong Content Security Policy (CSP) as an additional defense.

OWASP Reference

OWASP Top 10: A03 – Injection

Related vulnerability: Cross-Site Scripting (XSS)

Conclusion

The lab demonstrates how attacker-controlled URL fragments can become dangerous when they are passed into a DOM sink such as a jQuery selector. Proper input validation and safe DOM manipulation are required to prevent DOM-based XSS.

GitHub Commit Message

Add DOM XSS jQuery hashchange report